End-to-End Data Management Pipeline for a Book Recommendation System

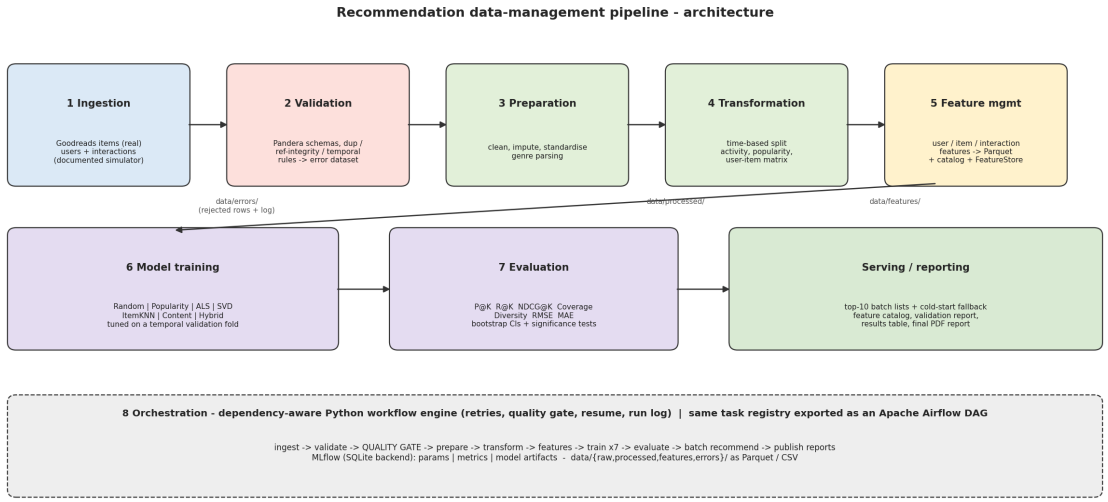
Data Management for Machine Learning - Assignment 2 - Final report

1. Problem and data

An online book platform wants to recommend books a reader is likely to rate ≥ 4 in the **future**. The pipeline ingests user, item and interaction data, validates it, prepares and transforms it, manages reusable features, trains and evaluates models and automates the whole workflow. **Items** are real: the Goodreads books catalogue (20,068 rows). Because the file has no reader-level data, **users** (4,015 rows) and **interactions** (126,106 rows, timestamped 2019-2024) are **synthetic and documented**: books are chosen proportionally to real popularity and taste match, and every rating is drawn from the book's *real* 1-5 star histogram. About 2% of the rows carry injected faults (duplicates, invalid ratings, orphan keys, bad timestamps) so that validation can be demonstrated and verified against a ledger. Model scores therefore describe the simulator, **not** real Goodreads behaviour; the engineering is what transfers.

2. Pipeline design and tools

Stage	Tool / module	Key design decision
Ingestion	pandas, src/ingestion	Raw extracts persisted as untouched CSV (like an upstream export); everything read as text so nothing is silently coerced.
Validation	Pandera + custom rules, src/validation	Rules have severity: *reject* (-> error dataset) or *warning* (repaired later). Users -> items -> interactions order enables referential-integrity checks.
Preparation	pandas, src/preparation	Category-median imputation with *_imputed* flags; genre boilerplate removed; repeated (user,item) pairs keep the latest.
Transformation	scipy.sparse, src/transformation	Time-based split at the 80th time-percentile (2024-01-17), asserted leak-free; 15 % temporal validation fold for tuning.
Feature mgmt	Parquet + FeatureStore class	Features computed from TRAIN only at a fixed REF_DATE; one class builds training and prediction features (train/serve consistency).
Training	implicit, Surprise, NumPy, MLflow	7 models incl. a popularity baseline; every trial is a nested MLflow run; models are plain-array objects.
Evaluation	src/evaluation	P/R/NDCG@5,10, Coverage, Diversity, RMSE, MAE; bootstrap CIs and paired significance tests over users.
Orchestration	Python DAG engine + Airflow DAG	One task registry drives both; retries, quality gate, resume/caching, run log and DAG-run graph.



3. Data validation and preparation

Table	Rows	Rejected	Warning rows	Main findings
users	4,015	20	69	duplicate user_id, future / unparsable signup_date; missing country, invalid age_group repaired
items (real)	20,068	684	7,110	near-empty rows, duplicate bookId; missing genres / pages / star histogram
interactions	126,106	2,772	0	bad dtype / range / null ratings, bad or future timestamps, orphan keys, duplicates, pre-signup events

Rejected rows are written to *data/errors/* (CSV + Parquet) with the violated rules; the full log and an HTML report are kept in *reports/*. The detection check confirms that **14 of 14** injected fault families were found. Validation also exposed genuine problems of the real file. Referential integrity cascades: rejecting a user orphans all of that user's interactions, which is why validation order matters.

4. Feature management

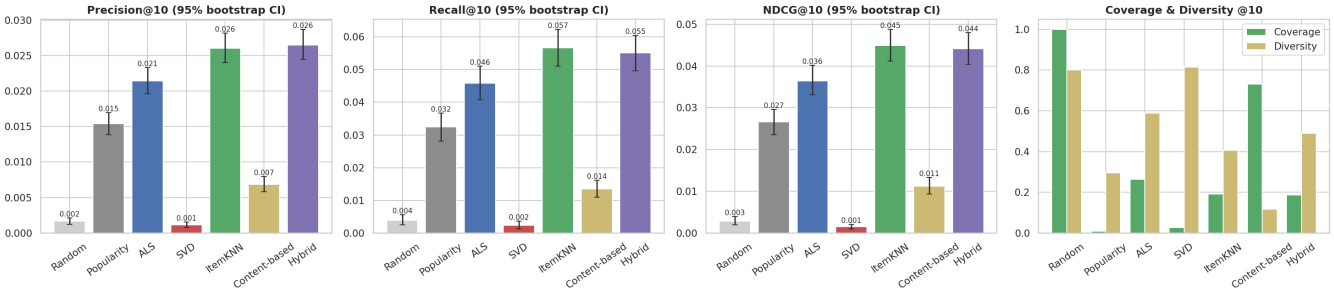
The feature catalog (25 documented entries: 10 user, 11 item, 4 interaction) lists name, type, definition, source and leakage guard (*reports/feature_catalog.csv|md*). Examples: user average rating, activity count, preferred category, days since last interaction; item popularity, Bayesian average rating, top audience age group; user-category affinity and item-vs-user rating gap. Cold-start users (179 of 3,401 test users) have no features by construction and are served a popularity fallback.

5. Models and results

Model	Precision@10	Recall@10	NDCG@10	Coverage@10	Diversity@10	RMSE	MAE
Random (sanity floor)	0.0017	0.0040	0.0028	1.0000	0.8009	n/a	n/a
Popularity (baseline)	0.0154	0.0324	0.0266	0.0080	0.2960	0.9336	0.7540
ALS (implicit MF)	0.0214	0.0458	0.0365	0.2644	0.5886	n/a	n/a
SVD (Surprise MF)	0.0011	0.0024	0.0015	0.0263	0.8150	0.9317	0.7316
ItemKNN (cosine CF)	0.0260	0.0567	0.0449	0.1907	0.4050	n/a	n/a
Content-based (genres)	0.0069	0.0135	0.0112	0.7312	0.1168	n/a	n/a
Hybrid (ItemKNN + ALS + content)	0.0265	0.0550	0.0442	0.1871	0.4888	n/a	n/a

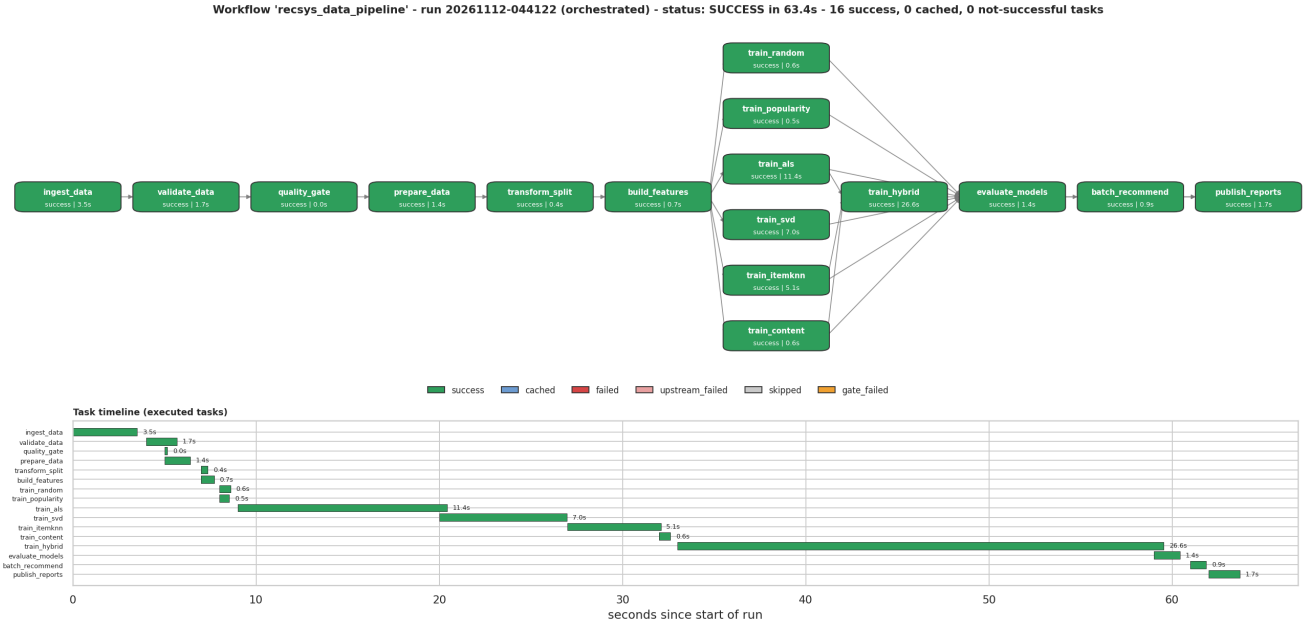
Test window, evaluated on 3,061 users with at least one relevant test item; K = 10; best personalised model in bold.

ItemKNN (cosine CF) is the best ranker (NDCG@10 +69% vs. the popularity baseline). The hybrid is statistically tied with ItemKNN on NDCG@10 (paired bootstrap 95 % CI [-0.0035, +0.0021]) but gives more diverse lists (diversity 0.49 vs. 0.40). SVD improves rating error only marginally (RMSE 0.932 vs. 0.934) and, used as a ranker, falls below random: rating accuracy and top-K usefulness are different objectives. Popularity covers 0.8% of the catalogue, personalised models far more.



6. Orchestration and reproducibility

The workflow has 16 tasks with explicit dependencies (ingest -> validate -> **quality gate** -> prepare -> transform -> features -> 7 training tasks -> evaluate -> batch recommend -> publish). Latest run *20261112-044122*: **success**, 16/16 tasks successful, 63 s. The gate stops the pipeline (downstream tasks are *skipped*) if more than 5% of interaction rows are rejected; failed tasks are retried; *resume* mode skips tasks that already succeeded with an identical configuration. All seeds are fixed, and the metrics of the orchestrated run are compared with the stage-by-stage run in the notebook. The same task registry is exported as an Apache Airflow DAG (*dags/recommendation_pipeline_dag.py*).



7. Limitations and future work

- **Simulated users / interactions:** conclusions about model ranking hold for this simulator; the pipeline itself transfers to real logs.
- **Editions:** different editions of one book have separate ids (e.g. several Harry Potter editions in the popularity list); entity resolution to a work id is needed in production.
- **Offline evaluation only:** one temporal split, no A/B test; absolute scores are low because the matrix is ~99 % sparse and only the future window is scored.
- **Content features** are used only through the genre profile in the hybrid; richer text / author embeddings, cold-start modelling and a hosted feature store / MLflow server are natural extensions.